

RISC-V CPU 设计文档

一、总体设计方案

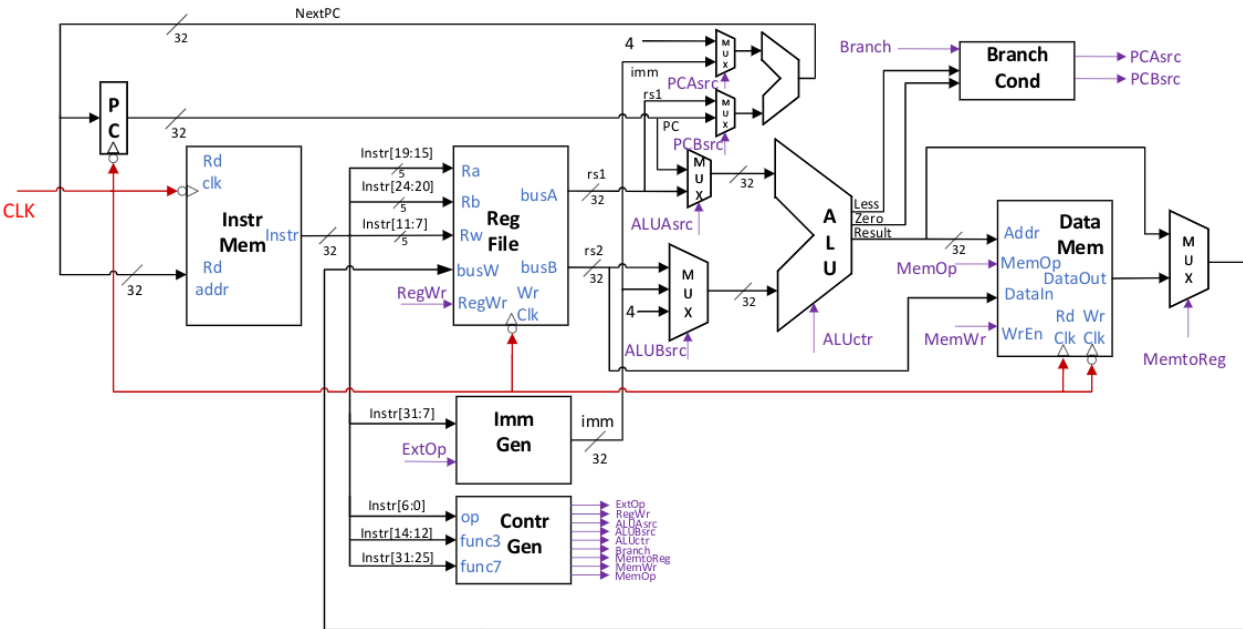
本设计实现了一个 32 位 RISC-V 多周期 CPU，并在阶段划分上参考了经典五级流水线思想。CPU 将指令存储器 IM 与数据存储器 DM 分开建模；指令执行由 ControlUnit 有限状态机分周期推进，数据通路由 PC、IR、寄存器堆、立即数生成器、ALU、数据存储器和写回选择器组成。

与单周期 CPU 相比，多周期结构将“取指、译码、执行、访存、写回”拆分到不同周期完成，便于复用 ALU 和存储访问通路，也便于在每个阶段插入寄存器保存中间结果。本工程中使用 IR 保存当前指令，使用 A/B 锁存寄存器堆读出的源操作数，使用 ALUOut 保存 ALU 结果。

CPU 对外通过 miniRV_SoC 顶层封装，输入为 fpga_clk 与 fpga_rst，输出为 debug_wb_have_inst、debug_wb_pc、debug_wb_ena、debug_wb_reg、debug_wb_value 等调试提交接口，用于测试框架观察每条指令的提交结果。

二、系统架构与模块划分

CPU 顶层 riscv.v 负责例化并连接各个子模块。Decode 进行组合译码，ControlUnit 负责状态转移和时序使能，ALU 完成算术逻辑与比较，NPC 计算下一条 PC，RF、IM、DM 分别实现寄存器堆、指令存储器与数据存储器。整体结构如图所示。



文件/模块	功能定位	说明
miniRV_SoC.sv	SoC 顶层封装	连接 FPGA 时钟/复位与 debug_wb_* 调试端口，内部例化 riscv CPU。
riscv.v	CPU 顶层数据通路	连接 PC、NPC、IM、IR、Decode、ControlUnit、RF、ALU、DM、写回选择器与调试提交逻辑。
ControlUnit.v	控制器/状态机	根据指令类型、分支标志和当前状态产生 PCWrite、IRWrite、RFWrite、DMCtrl、NPCOp 等时序控制信号。
Decode.v	组合译码器	由 opcode、Funct3、Funct7 产生指令分类、ALUOp、ALUSrc、WDSel、RegSel、MemRead/MemWrite、RFWE 与默认 NPCOp。

文件/模块	功能定位	说明
ALU.v	算术逻辑单元	支持 ADD、SUB、AND、OR、XOR、SLL、SRL、SRA、SLT、SLTU，并输出 zero、less、lessu。
RF.v	寄存器堆	32 个 32 位通用寄存器；x0 恒为 0；读为组合输出，写在时钟上升沿进行。
IM.v	指令存储器	8192 x 32 位指令存储，按 PC[14:2] 取指，可通过 PATH 宏初始化。
DM.v	数据存储器	8192 x 32 位数据存储，支持字节/半字/字读写，并完成符号扩展或零扩展。
ImmGen.v	立即数生成器	生成 I、S、B、U、J 型立即数，以及 I 型零扩展立即数。
NPC.v	下一 PC 生成	支持 PC+4、PC+ImmB、rs1+ImmI、PC+ImmJ 等下一地址选择。
PC.v、IR.v、Flopr.v	时序寄存器	PC、指令寄存器与通用带使能锁存器，用于多周期阶段间保存数据。
MUX_*.v	多路选择器	实现写寄存器号选择、ALU 输入选择和写回数据选择。

三、指令集说明

本 CPU 实现 RISC-V RV32I 的常用整数指令子集，覆盖算术逻辑、移位、比较、访存、条件分支、无条件跳转和 U 型立即数指令。指令分类依据 opcode，具体操作由 Funct3/Funct7 进一步区分。

指令类型	支持指令	实现说明
R 型算术逻辑	add、sub、and、or、xor、sll、srl、sra、slt、sltu	寄存器 rs1 与 rs2 作为 ALU 输入，结果写回 rd。
I 型算术逻辑	addi、andi、ori、xori、slti、sltiu、slli、srli、srai	rs1 与 I 型立即数作为 ALU 输入；移位类使用零扩展立即数，其他 I 型使用符号扩展。
Load 类	lb、lh、lw、lbu、lhu	ALU 计算地址 rs1+imm，DM 根据 Funct3 和地址低位读出字节/半字/字并扩展后写回 rd。
Store 类	sb、sh、sw	ALU 计算地址 rs1+imm，DM 根据 Funct3 和地址低位将 rs2 的低 8/16/32 位写入存储器。
B 型条件分支	beq、bne、blt、bge、bltu、bgeu	ALU/比较标志产生 zero、less、lessu，ControlUnit 按 Funct3 判断是否采用 PC+ImmB。
J 型跳转	jal	写回 PC+4 到 rd，下一 PC 为 PC+ImmJ。
I 型跳转	jalr	写回 PC+4 到 rd，下一 PC 为 (rs1+ImmI)&~1。
U 型	lui、auipc	lui 写回 ImmU；auipc 写回 PC+ImmU。

格式	字段结构	用途
R 型	funct7 rs2 rs1 funct3 rd opcode	用于寄存器-寄存器 ALU 指令。
I 型	imm[11:0] rs1 funct3 rd opcode	用于 I 型 ALU、load、jalr；load/jalr 使用符号扩展立即数。
S 型	imm[11:5] rs2 rs1 funct3 imm[4:0] opcode	用于 store 指令，立即数由 ImmGen 拼接并符号扩展。
B 型	imm[12 10:5] rs2 rs1 funct3 imm[4:1 11] opcode	用于条件分支，最低位补 0，形成 2 字节对齐偏移。
U 型	imm[31:12] rd opcode	用于 lui/auipc，低 12 位补 0。
J 型	imm[20 10:1 11 19:12] rd opcode	用于 jal，最低位补 0。

四、关键技术说明

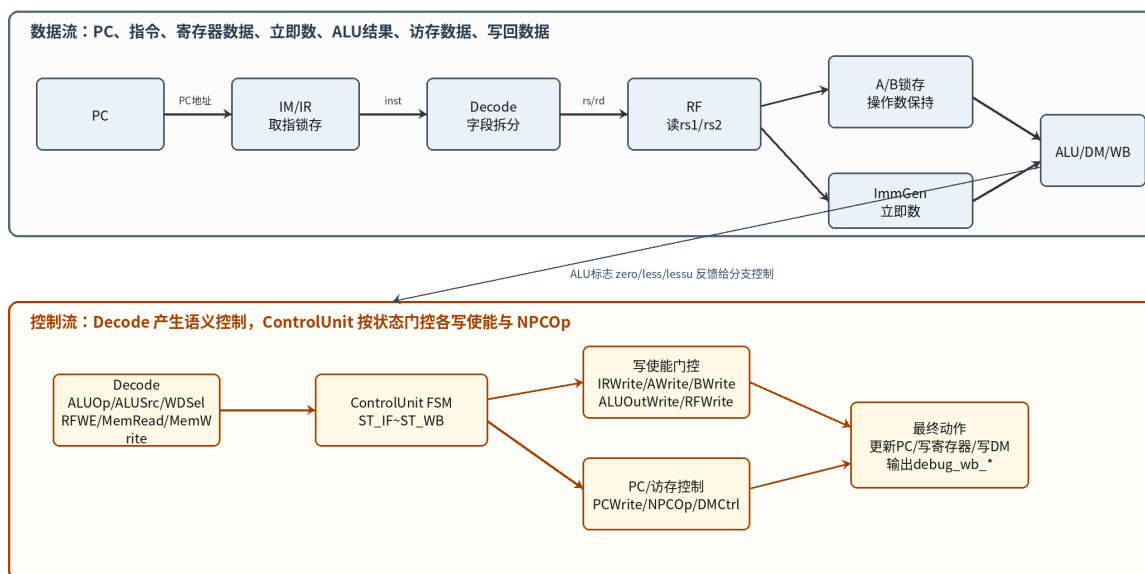
关键技术	说明
1. 译码与时序控制分离	Decode 只根据指令字段产生组合控制信息；ControlUnit 再根据状态机节拍门控 PC、IR、RF、ALUOut、DM 等写使能，使控制逻辑层次清晰。
2. 多周期中间寄存器	IR 保存当前指令，A/B 保存寄存器堆输出，ALUOut 保存执行结果，避免组合路径跨越过长，也使每个阶段可以复用同一套 ALU。
3. 立即数统一生成	ImmGen 同时生成 I/S/B/U/J 五种立即数，并额外提供 ImmIZero，以适配移位立即数的零扩展需求。
4. 分支比较机制	ALU 输出 zero、less、lessu。ControlUnit 在 ST_EX_BR 状态依据 Funct3 判断 beq/bne/blt/bge/bltu/bgeu 是否跳转，并选择 NPCOp。
5. 访存宽度与符号扩展	DM 通过 Addr[1:0] 定位字内字节/半字，根据 Funct3 完成 lb/lh 的符号扩展和 lbu/ldu 的零扩展；写入时支持 sb/sh/sw 局部更新。
6. 下一 PC 生成	NPC 模块统一处理顺序执行、条件分支、jal、jalr。jalr 地址最低位清零，符合 RISC-V 对 jalr 目标地址的要求。
7. 调试提交接口	riscv.v 使用 commit_now 在写回、访存写、分支和跳转等完成状态产生 debug_wb_*, 方便测试框架判断指令提交。

五、数据流 / 控制流设计

数据流从 PC 开始，经 IM 取出指令并写入 IR；Decode 拆分 opcode、Funct3、Funct7、rs1、rs2、rd 等字段；RF 读出两个源操作数并在 ID 阶段锁存到 A/B；ImmGen 产生所需立即数；ALU 完成运算、地址计算或比较；Load 指令通过 DM 读数据后写回，Store 指令在 MEM_WR 状态写入 DM，普通 ALU 指令在 WB_ALU 状态写回寄存器。

控制流由 Decode 和 ControlUnit 两级组成。Decode 产生指令“语义级”控制信号，例如 ALUOp、ALUSrcA/B、WDSel、RegSel、MemRead/MemWrite、RFWE 和默认 NPCOp；ControlUnit 根据状态机决定这些控制信号在哪个周期真正生效，例如 IRWrite 只在 IF 阶段有效，RFWrite 只在写回或跳转提交阶段有效，DMCtrl_WR 只在 Store 的 MEM_WR 阶段有效。

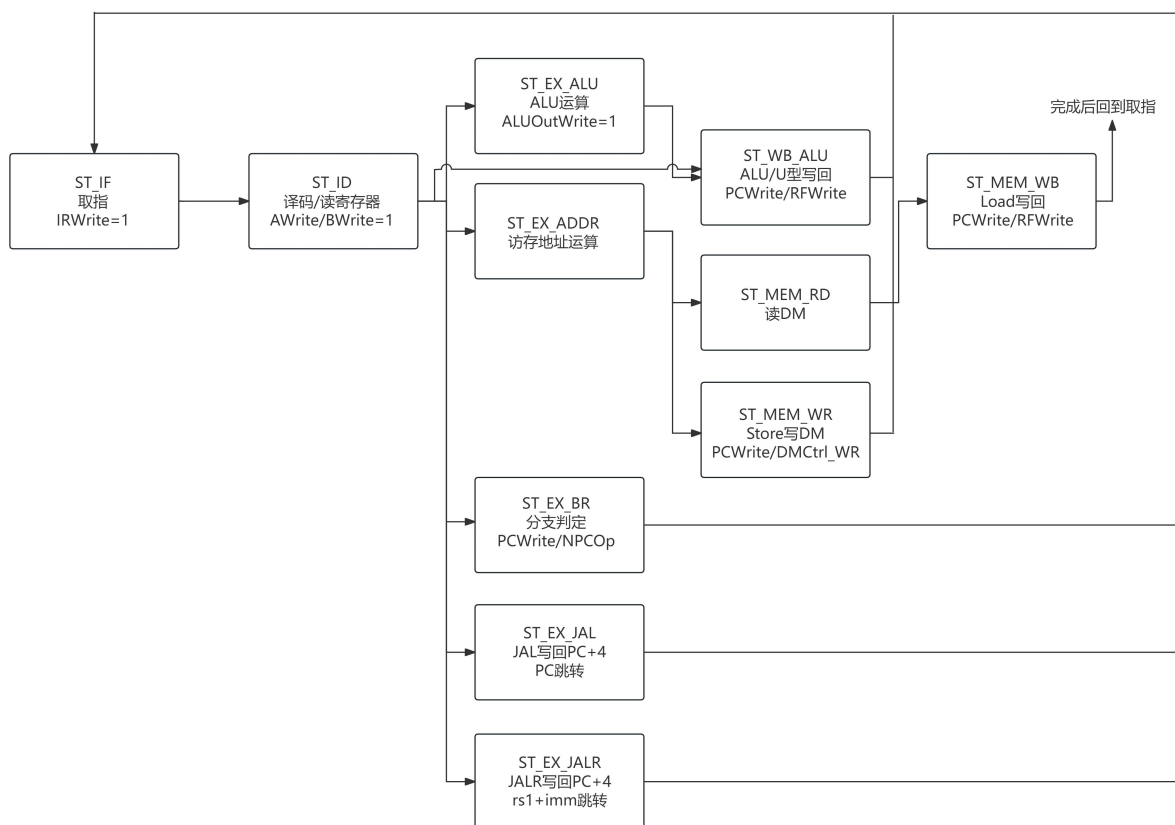
数据流 / 控制流设计框图



路径	数据流	控制要点
取指路径	PC -> IM -> IR	InsMemRW=1, IRWrite=1, 当前 PC 对应指令进入 IR。
译码/读寄存器路径	IR -> Decode ; IR.rs1/rs2 -> RF -> A/B	AWrite=1, BWrite=1, 源操作数在 ID 阶段锁存。
ALU 路径	A/B 或 Imm -> ALU -> ALUOut	ALUSrcA/B 选择操作数；ALUOp 决定运算；ALUOutWrite 锁存结果。
Load 路径	ALUOut 地址 -> DM -> WDSel -> RF	DM 组合读出 RD, WDSel_FromMEM 写回寄存器。
Store 路径	ALUOut 地址 + B 数据 -> DM	MEM_WR 阶段 DMCtrl_WR=1, 在时钟上升沿写 DM。
分支/跳转路径	ALU 标志/rs1/立即数 -> NPC -> PC	ControlUnit 选择 NPCOp, PCWrite 更新 PC。

六、控制状态机与指令执行周期

ControlUnit 采用显式多周期状态机。所有指令先进入 ST_IF 和 ST_ID，随后根据指令类型进入不同执行路径。普通 R/I 型指令执行 ALU 后写回；Load/Store 先计算地址，再进行访存；Branch 在 EX_BR 完成比较和 PC 更新；JAL/JALR 在对应执行状态完成 PC+4 写回和 PC 跳转；LUI/AUIPC 可从 ID 后直接进入写回状态。



指令类别	状态序列	说明
R/I 型 ALU	ST_IF -> ST_ID -> ST_EX_ALU -> ST_WB_ALU	EX_ALU 锁存 ALU_result；WB_ALU 写回 rd 并 PC=PC+4。
Load 类	ST_IF -> ST_ID -> ST_EX_ADDR -> ST_MEM_RD -> ST_MEM_WB	EX_ADDR 计算有效地址；MEM_RD 读 DM；MEM_WB 写回 rd 并更新 PC。
Store 类	ST_IF -> ST_ID -> ST_EX_ADDR -> ST_MEM_WR	EX_ADDR 计算有效地址；MEM_WR 写 DM 并更新 PC。
Branch 类	ST_IF -> ST_ID -> ST_EX_BR	EX_BR 根据 Funct3 与 zero/less/lessu 判断是否采用分支目标，并更新 PC。

指令类别	状态序列	说明
JAL	ST_IF -> ST_ID -> ST_EX_JAL	写回 PC+4, NPCOp=NPC_Offset20, PC 跳到 PC+ImmJ。
JALR	ST_IF -> ST_ID -> ST_EX_JALR	写回 PC+4, NPCOp=NPC_rs, PC 跳到 (rs1+ImmI)&~1。
LUI/AUIPC	ST_IF -> ST_ID -> ST_WB_ALU	WDSel_Else 选择 UTypeWD；lui 写 ImmU, auipc 写 PC+ImmU。

七、子模块时序说明

子模块	时序类型	时序/功能说明
PC	时序寄存器, posedge clk 或 posedge rst	rst=1 时 PC 清零；PCWrite=1 时在上升沿写入 NPC。
IR	时序寄存器, posedge clk	IRWrite=1 时锁存 IM 输出的 in_ins。
RF	读组合, 写时序	RR1/RR2 组合读；RFWrite=1 且 WR!=0 时在上升沿写寄存器；x0 始终保持 0。
Flopr(A/B/ALUOut)	带使能寄存器, posedge clk 或 rst	AWrite/BWrite/ALUOutWrite 有效时锁存对应输入, rst 时清零。
ALU	组合逻辑	输入 A、B 或控制 ALUOp 改变后立即组合产生 ALU_result、zero、less、lessu。
DM	读组合, 写时序	读由 Addr、Funct3 组合产生 RD；DMCtrl_WR=1 时在上升沿按字节/半字/字写入。
IM	组合读	当 InsMemRW=1 时输出 memory[PC[14:2]]，否则输出 0。
NPC	组合逻辑	根据 NPCOp、PC、立即数和 rs1 组合生成 NPC 与 PCA4。
Decode	组合逻辑	由 opcode/Funct3/Funct7 组合产生指令分类和控制信号。
ControlUnit	状态转移时序, 输出组合	上升沿更新状态；组合输出当前状态对应的写使能、DMCtrl 和 NPCOp。

八、主要控制信号说明

控制信号	作用	说明
PCWrite	PC 写使能	在指令完成或跳转/分支完成时更新 PC。
InsMemRW	指令存储器读使能	ST_IF 阶段为 1，用于输出当前 PC 对应指令。
IRWrite	IR 写使能	ST_IF 阶段锁存指令。
AWrite/BWrite	A/B 操作数锁存使能	ST_ID 阶段保存 RF 读出的 RD1/RD2。
ALUOutWrite	ALU 结果锁存使能	R/I 执行或地址计算阶段保存 ALU_result。
RFWrite	寄存器堆写使能	写回状态或 JAL/JALR 状态写 rd。
DMCtrl	数据存储器读/写控制	Load 为读，Store 的 MEM_WR 阶段为写。
NPCOp	下一 PC 选择	00: PC+4；01: PC+ImmB；10: rs1+ImmI；11: PC+ImmJ。
ALUOp	ALU 操作选择	选择 ADD/SUB/AND/OR/XOR/SLL/SRL/SRA/SLT/SLTU 等运算。
ALUSrcA/B	ALU 输入选择	A 端可选寄存器数据或 0；B 端可选寄存器数据、普通立即数或地址立即数。
WDSel	写回数据选择	选择 ALUOut、DM 读数据、PC+4 或 UTypeWD。
RegSel	写寄存器编号选择	当前实现主要选择 rd，也保留 x0、x31 等接口形式。

九、访存设计说明

数据存储器 DM 以 32 位字为基本存储单元，通过 Addr[14:2] 定位字地址，通过 Addr[1:0] 定位字内字节偏移。对于 lb/lbu，DM 按偏移选择 8 位数据；对于 lh/lhu，依据 Addr[1] 选择低半字或高半字；对于 lw，直接返回整字。

写操作在时钟上升沿完成。sb 只更新目标字中的一个字节，sh 更新低半字或高半字，sw 更新整个 32 位字。初始化时 IM 与 DM 均可通过 PATH 宏从二进制文件读取，并在读取后进行字节序重排，以匹配测试框架的指令/数据存储格式。

指令	Funct3	DM 行为
lb	3'b000	读取 8 位并符号扩展到 32 位
lh	3'b001	读取 16 位并符号扩展到 32 位
lw	3'b010	读取 32 位字
lbu	3'b100	读取 8 位并零扩展到 32 位
lhu	3'b101	读取 16 位并零扩展到 32 位
sb	3'b000	写 WD[7:0] 到目标字节
sh	3'b001	写 WD[15:0] 到目标半字
sw	3'b010	写 WD[31:0] 到目标字

十、顶层调试接口设计

miniRV_SoC 将 CPU 的调试信号直接引出。riscv.v 内部使用 commit_now 表示当前周期是否有一条指令完成。对于 R/I 写回、Load 写回、Store 写存储器、Branch、JAL/JALR，以及非法指令识别等情况，commit_now 置位，并把当前 PC、写回使能、目标寄存器和写回值送到 debug_wb_*。

信号	含义
debug_wb_have_inst	当前周期有指令完成/提交
debug_wb_pc	提交指令对应 PC
debug_wb_ena	提交周期是否写寄存器
debug_wb_reg	写回寄存器编号
debug_wb_value	写回数据值

十一、附加要求

在完成基本 CPU 功能实现的基础上，本设计进一步引入了流水线优化思想，将指令执行过程划分为取指、译码、执行、访存和写回等相对独立的阶段，使多条指令能够在不同阶段并行推进。相比单周期或普通多周期 CPU 每次只完整执行一条指令的方式，流水线结构能够提高硬件资源的利用率，减少指令之间的空闲等待时间，从而提升整体指令吞吐率。设计中通过阶段寄存器保存各流水级之间的中间结果和控制信号，使 PC、指令寄存器、寄存器堆、ALU、数据存储器和写回通路能够按照时钟节拍协同工作。流水线优化实现了 CPU 结构从“顺序执行”向“并行执行”的改进，也使本设计更接近现代处理器的基本实现方式。

十一、总结与改进方向

本设计以多周期方式实现 RV32I 常用整数指令子集，模块划分清晰，控制器与数据通路分离，能够覆盖 ALU、Load/Store、Branch、Jump 和 U 型指令的基本执行流程。通过 A/B/ALUOut 等中间寄存器，CPU 在不同阶段复用 ALU 与存储通路，降低了单周期组合路径压力。

后续可以进一步改进：第一，完善非法指令异常处理与 PC 更新策略；第二，将 IM/DM 接口扩展为总线接口；第三，增加更完整的形式化验证或随机指令测试，提高设计可靠性。